Report of CPT205 Assessment 1

0. Basic Information

Module Code/Title	CPT205 Computer Graphics
Name	Jiaqi Wang
Student ID	2362441
Degree Programme	Information and Computing Science

1. Design Overview & Features

The greeting card project for the university 20th anniversary is divided into three interactive parts. The first part shows the building of our school, representing the outside environment. User can interact with this scene by pressing specific keys on the keyboard to transition to the second part: a dark room illuminated only by a single flame on candle. By performing another keyboard interaction, users proceed to the final part: a bright room featuring a large birthday cake surrounded by colorful balloons. In this scene, user can further engage by using the mouse to control the rising speed of balloons, and they will also see the written congratulatory message displayed in the center of the screen.

2 Geometry & Modelling

2.1 Reusable Primitives:

windowLine(float x, float y, float w, float h): This function draws a diagonal reflection line inside a window to simulate the effect of glass reflection.

Several reusable primitive functions are implemented to draw both the filled and outlined versions of basic geometric shapes, including circles, ellipses, rectangles. (eg. **drawUnitCircleOutline()** and **drawUnitCircleFilled()**) These functions serve as modular building blocks for constructing complex objects.

2.2 Objects:

2.2.1 Buildings

The campus buildings and the street are constructed from reusable 2D primitives with simple transforms. Rectangular walls and doors are drawn with **GL_QUADS** via **rectFilled()**, while contours use **GL_LINE_LOOP** via **rectOutline()**. Window grids are generated procedurally by **windowColumn()**, which repeats filled rectangles and outlines, and adds a highlight line to suggest glass reflection. Each block (left wing, right wing, the central building and gressland) is assembled by **drawWingCore()**.The street and ground are rendered as a tapered quadrilateral with **quadFilled()**.

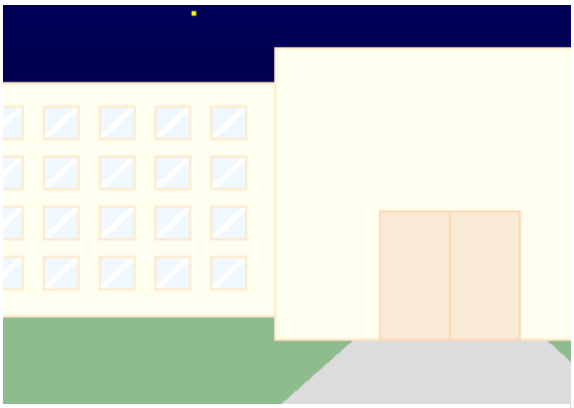


Figure 1 Building

2.2.2 Starry Sky

The starry background is created using point primitives. Small yellow points are rendered with **GL_POINTS** and **glVertex2f()** inside a single **glBegin(GL_POINTS)** block. Each point represents a star randomly positioned across the upper sky area. The background of the outdoor scene is drawn using a color gradient created with **GL_QUADS** (Similarly, the indoor scene applies the same gradient technique but with warmer colors).

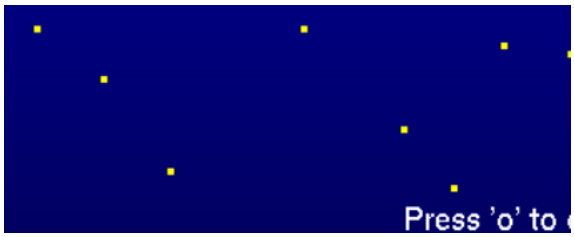


Figure 2 Starry Sky

2.2.3 Cake & Plate & Candle

The plate and cake are modeled using combinations of ellipses and rectangles to create a layered 3D appearance. The base plate is drawn with **drawEllipseFilled()** and **drawEllipseOutline()** to represent its body and rim. The cake itself is constructed by stacking multiple ellipses and rectangles of slightly different sizes and colors. Filled ellipses **(GL_TRIANGLE_FAN)** are used to represent the curved top and bottom layers, while **rectFilled()** and **rectOutline()** form the side walls of each tier, giving the cake realistic height and thickness. A small rectangular candle (**rectFilled**) is added on top, with a short line segment as the wick.

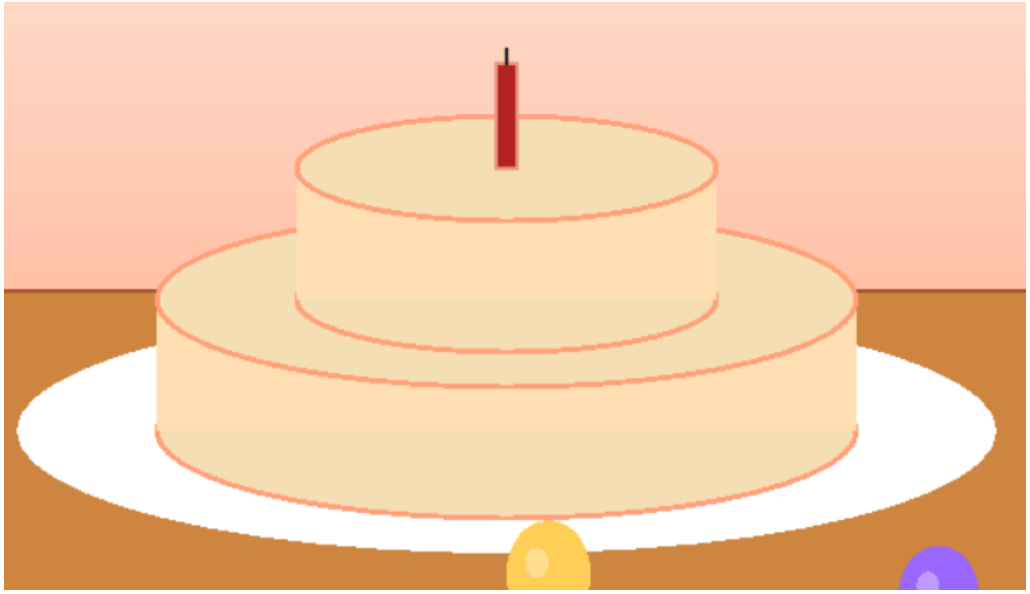


Figure 3 Cake & Plate & Candle

2.2.4 Candlelight

The candlelight is composed of several overlapping ellipses with different colors and transparency levels to simulate the gradual glow of a real flame. Each ellipse is drawn using **GL_TRIANGLE_FAN** in the **drawCandleFlame()** function. This layered approach visually enhances the flame's depth and warmth, making it appear gently flickering against the dark background.

Figure 4 Candlelight

2.2.5 Balloons

Each balloon's main body is drawn with a filled ellipse created through **GL_TRIANGLE_FAN**. A small triangle is drawn beneath the ellipse to represent the tied balloon end **(GL_TRIANGLES)**, a vertical line **(GL_LINES)** connects it to simulate the string.

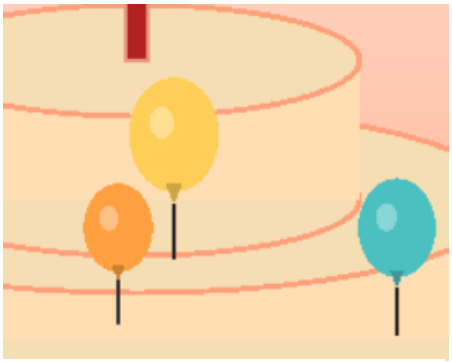


Figure 5 Balloons

3. Further function

3.1 Transformations & Viewing

Projection: The entire scene is displayed through a fixed 2D orthographic projection defined by **glOrtho(-400, 400, -300, 300, -1, 1)**. This projection ensuring all objects are drawn within the same 800×600 logical space.

Local transforms. Each graphical object is constructed in its own local coordinate system and positioned in the scene using transformation operations. The functions **glPushMatrix()**, **glTranslatef()**, **glScalef()**, and **glPopMatrix()** are combined to apply translation and scaling locally.

3.2 Animation and Control

The motion of the balloons is animated through a timer function implemented with **glutTimerFunc()**. Each frame, the timer updates the vertical position of every balloon by adding its velocity multiplied by the global speed scale. When a balloon moves beyond the upper boundary of the scene, it is repositioned below the visible area to create a continuous floating effect. The display is refreshed after each update, producing smooth upward movement.

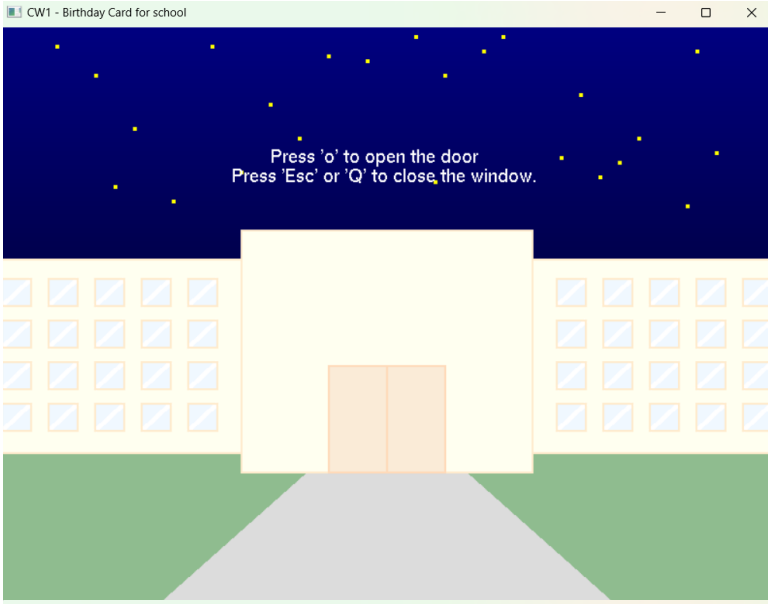
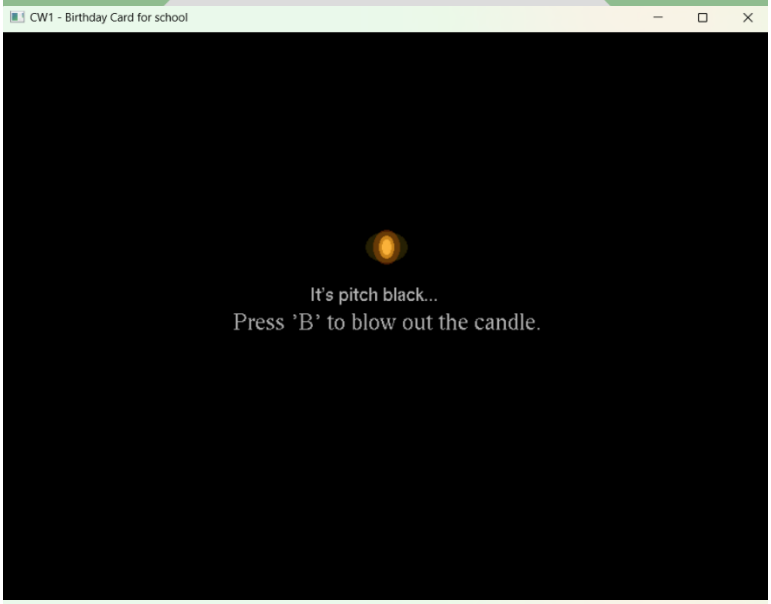
Mouse input is handled by the function **mouse_input()**, where button events modify the variable **g_speedScale**. The left, right, and middle mouse buttons adjust this multiplier by decreasing, increasing, or resetting its value. **Keyboard input** in **keyboard_input()** changes the variable **g_scene**, switching program states between **SCENE_OUTDOOR**, **SCENE_BLACK**, and **SCENE_INDOOR**

4. Interaction (Keyboard & Mouse)

Keyboard:

‘o’ or ‘O’	Open the door → Black Field (Transition)
‘b’ or ‘B’	Enter the indoor scene
‘’g or ‘G’	Back to the outdoor scene
‘q’ or ‘Q’ or ‘Esc’	Exit the program

Mouse:

Left	Slow down balloon
Right	Speed Up Balloons
Middle	Reset the balloon speed

5. Screenshots